

Facilitator answer key

Match the Git command to the GUI action

Answer key

Command	Answer	GUI action
1. <code>git clone <url></code>	D	File > New Project > Version Control > Git
2. <code>git add / stage</code>	F	Tick the checkbox next to a file in the Git pane
3. <code>git commit -m "..."</code>	B	Write a message, click Commit
4. <code>git push</code>	E	Up-arrow (push) button in the Git pane
5. <code>git pull</code>	A	Down-arrow (pull) button in the Git pane
6. <code>git status</code>	H	Read the file list and status flags in the Git pane
7. <code>git log</code>	K	History (clock icon) in the Git pane
8. <code>git checkout -b <name></code>	I	Branch dropdown > New Branch
9. <code>git revert <commit></code>	J	Right-click a commit in History > Revert
10. <code>git add + commit</code> on GitHub	C	Add file / Upload files, then Commit changes
11. open a pull request (<i>GitHub feature</i>)	G	Pull requests tab > New pull request

Talking points while reviewing

- `git add` and `git commit` are two steps. In the RStudio pane you see this clearly: tick the box (add), then Commit. On GitHub the web editor bundles both into one “Commit changes” click.
- `git revert` makes a new commit that undoes an earlier one; it does not erase history. In the GUI this is a right-click on a commit in History. Good moment to contrast it with deleting or force-pushing.
- A pull request is not a Git command in the same sense; it is a GitHub feature built on top of branches you pushed. That is why it is called out separately on the sheet. Good moment to name where Git ends and GitHub begins.
- Tie-in to the agentic follow-up: an agent driving Git speaks these same command names. Knowing the mapping is what lets you read and check what the agent does.